

Networking Essentials

Phase 1 · Fundamentals · Estimated time: 2 hours

1. Concept From First Principles

English:

Every network conversation needs an agreed-upon set of rules -- a protocol -- so both sides know how to interpret the bytes flowing between them. This chapter covers the handful of protocols that show up in almost every HLD interview: TCP, UDP, HTTP/HTTPS, and WebSockets.

Hindi:

हर network बातचीत को नियमों के एक तय सेट की ज़रूरत होती है -- एक protocol -- ताक दोनों तरफ पता हो कि आपस में आने-जाने वाले bytes को कैसे समझना है। यह chapter उन कुछ protocols को cover करता है जो लगभग हर HLD interview में आते हैं: TCP, UDP, HTTP/HTTPS, और WebSockets।

Important Words:

- **Protocol** – प्रोटोकॉल / नियमों का सेट

TCP vs UDP -- the foundational trade-off

English:

Both protocols sit at the same layer directly above IP, and everything above them -- HTTP, gRPC, WebSockets, WebRTC -- is built on top of one or the other. Choosing TCP or UDP is often an implicit choice made by whichever higher-level protocol you pick.

Hindi:

दोनों protocols IP के ठीक ऊपर एक ही layer पर होते हैं, और इनके ऊपर बना हर protocol -- HTTP, gRPC, WebSockets, WebRTC -- इन्हीं दोनों में से किसी एक पर बना होता है। TCP या UDP चुनना अक्सर एक implicit choice होता है जो आपके चुने हुए higher-level protocol से पहले ही तय हो जाता है।

Important Words:

- **Implicit** – अंतर्नहिती / परीक्षण

English:

TCP (Transmission Control Protocol) is connection-oriented and reliable: before any data flows, client and server perform a three-way handshake. Every packet is acknowledged, lost packets are retransmitted, and packets are reassembled in the correct order. This reliability costs latency and overhead.

Hindi:

TCP (Transmission Control Protocol) connection-oriented और reliable होता है: कोई भी data भेजने से पहले, client और server एक three-way handshake करते हैं। हर packet को acknowledge किया जाता है, खो गए packets दोबारा भेजे जाते हैं, और packets सही क्रम में जोड़े जाते हैं। यह reliability अपने साथ latency और overhead लाती है।

Important Words:

- **Reliable** – विश्वसनीय / भरोसेमंद

- **Acknowledged** – स्वीकार किया गया
- **Retransmitted** – दोबारा भेजा गया
- **Overhead** – अतिरिक्त बोझ / ओवरहेड

English:

UDP (User Datagram Protocol) is connectionless: the sender fires packets without waiting for acknowledgment. No handshake, no guaranteed delivery, no guaranteed order -- but much lower latency and overhead.

Hindi:

UDP (User Datagram Protocol) connectionless होता है: sender बिना acknowledgment का इंतज़ार किए packets भेज देता है। कोई handshake नहीं, delivery की कोई guarantee नहीं, order की कोई guarantee नहीं -- लेकिन latency और overhead बहुत कम।

Important Words:

- **Connectionless** – कनेक्शनलेस / बिना कनेक्शन के

Backend engineer analogy

English:

TCP is a phone call -- you dial, wait for pickup, confirm they can hear you. UDP is shouting an announcement across a room -- fast, no confirmation anyone heard it. This is why video calls often use UDP-based protocols -- a dropped video frame is fine, but waiting for it feels laggy.

Hindi:

TCP एक phone call जैसा है -- आप dial करते हैं, जवाब का इंतज़ार करते हैं, confirm करते हैं कि उन्हें सुनाई दे रहा है। UDP कमरे में जोर से कुछ announce करने जैसा है -- तेज़, लेकिन किसी ने सुना या नहीं इसकी कोई पुष्टि नहीं। इसीलिए video calls अक्सर UDP-based protocols use करती हैं -- एक frame छूट जाए तो चलेगा, लेकिन उसका इंतज़ार करना laggy महसूस होता है।

Important Words:

- **Laggy** – लैगी / धीमा और अटका हुआ

English:

A quick but important addition: HTTP/1.1 opens one connection per request by default, which historically caused 'head-of-line blocking.' HTTP/2 introduced multiplexing, letting many requests share a single connection simultaneously without blocking each other.

Hindi:

एक छोटी लेकिन ज़रूरी बात: HTTP/1.1 डिफ़ॉल्ट रूप से हर request के लिए एक अलग connection खोलता है, जिससे पहले 'head-of-line blocking' की समस्या होती थी। HTTP/2 ने multiplexing लाया, जिससे कई requests एक ही connection को एक साथ share कर सकते हैं, बिना एक-दूसरे को रोके।

Important Words:

- **Multiplexing** – मल्टीप्लेक्सिंग

HTTP and HTTPS

English:

HTTP is an application-layer protocol built on top of TCP. It defines methods (GET, POST, PUT, DELETE), status codes, and a request/response format. HTTPS is HTTP wrapped in TLS encryption -- the same model, but the bytes on the wire are encrypted.

Hindi:

HTTP एक application-layer protocol है जो TCP के ऊपर बना है। यह methods (GET, POST, PUT, DELETE), status codes, और एक request/response format define करता है। HTTPS वही HTTP है बस TLS encryption में लपेटा हुआ -- वही model, लेकिन wire पर जाने वाले bytes encrypted होते हैं।

Important Words:

- **Encryption** – एन्क्रिप्शन / गोपनीयकरण

WebSockets

English:

HTTP is fundamentally request-response: the client always speaks first. WebSockets establish a single persistent TCP connection that both client and server can send messages over, at any time, in either direction. This is the backbone of chat apps, live order tracking, and live score updates.

Hindi:

HTTP मूल रूप से request-response है: client हमेशा पहले बोलता है। WebSockets एक persistent TCP connection बनाते हैं जसि पर client और server दोनों कभी भी, कसि भी दशि में message भेज सकते हैं। यही chat apps, live order tracking, और live score updates की नींव है।

Important Words:

- **Persistent** – स्थायी / लगातार बना रहने वाला

2. Real-World Example / Case Study

English:

Careem's live driver-location tracking uses a persistent connection model so the rider's app receives location updates as they happen, instead of polling an HTTP endpoint every few seconds. Careem's payment confirmation API is a classic HTTPS request-response call since it doesn't need push updates.

Hindi:

Careem की live driver-location tracking एक persistent connection model use करती है ताकि rider के app को location updates तुरंत मिलें, बजाय इसके कि हर कुछ seconds में एक HTTP endpoint को poll किया जाए। Careem का payment confirmation API एक classic HTTPS request-response call है क्योंकि उसे push updates की ज़रूरत नहीं।

English:

Video/voice features in apps like Talabat's in-app chat-with-rider calls typically rely on UDP-based real-time transport because a slightly glitchy but current audio stream beats a perfectly reliable but delayed one.

Hindi:

Talabat जैसे apps में rider के साथ chat/call जैसे video/voice features आमतौर पर UDP-based real-time transport पर चलते हैं, क्योंकि थोड़ा glitchy लेकिन current audio, पूरी तरह reliable लेकिन देर से आने वाले audio से बेहतर होता है।

Important Words:

- **Glitchy** – गड़बड़ / अटकने वाला

English:

A third example: Razorpay's webhook system, which notifies merchants when a payment's status changes, is plain HTTPS request-response -- Razorpay's server acts as the client in that specific call. This is a useful reminder that client/server are roles, not fixed identities.

Hindi:

एक तीसरा उदाहरण: Razorpay का webhook system, जो merchants को payment status बदलने पर सूचति करता है, plain HTTPS request-response है -- इस specific call में Razorpay का server ही client की भूमिका निभाता है। यह याद दिलाता है कि client/server सिर्फ roles हैं, कोई fixed पहचान नहीं।

Important Words:

- **Notify** – सूचति करना

3. Diagram (English labels kept as-is)

TCP handshake (reliable, connection-oriented):		
Client	--- SYN ----->	Server
Client	<-- SYN-ACK -----	Server
Client	--- ACK ----->	Server
UDP (connectionless, no handshake):		
Client	--- datagram ----->	Server (fire and forget)
WebSocket (starts as HTTP, then upgrades):		
Client	--- HTTP GET + Upgrade header -->	Server
Client	<-- 101 Switching Protocols -----	Server
[persistent connection: EITHER side can send, anytime]		

4. Trade-offs Table (Reference Table – English)

Protocol	Best for	Avoid when
TCP	APIs, file transfer, anything needing correctness	Ultra-low-latency streaming
UDP	Video/voice calls, live game state, DNS queries	Anything requiring guaranteed delivery (payments!)
HTTP/HTTPS	Standard request-response APIs	True real-time, server-initiated push
WebSockets	Chat, live tracking, live notifications	Simple CRUD APIs

5. Common Interview Questions

Q1: "Would you use TCP or UDP for a live video streaming feature? Why?"

English:

Model approach: Choose UDP-based transport, explain that retransmitting a lost video frame after the fact is worse than simply skipping it, and that low latency matters more than perfect reliability here.

Hindi:

Model approach: UDP-based transport चुनें, बताएं कखि गया video frame बाद में दोबारा भेजना उसे छोड़ देने से भी बुरा है, और यहाँ perfect reliability से ज़्यादा low latency मायने रखती है।

Q2: "How would you design real-time order status updates without WebSockets?"

English:

Model approach: Describe short-interval polling or long polling as fallbacks, explain their cost versus WebSockets, and note that many production systems use polling as a pragmatic first version.

Hindi:

Model approach: short-interval polling या long polling को fallback के रूप में बताएं, WebSockets के मुकाबले इनकी लागत समझाएं, और यह भी बताएं कि कई production systems शुरुआत में polling को ही एक practical पहला version मानकर use करती हैं।

Important Words:

- **Pragmatic** – व्यावहारिक

Q3: "A webhook receiver is occasionally getting duplicate events -- why, and how do you handle it?"

English:

Model approach: Explain that most webhook senders use 'at-least-once' delivery -- if they don't receive a timely 200 OK, they retry, causing duplicates. The fix is making the receiver idempotent: track processed event IDs and skip ones already handled.

Hindi:

Model approach: बताएं कि ज़्यादातर webhook senders 'at-least-once' delivery use करते हैं -- अगर उन्हें समय पर 200 OK ना मिले, तो वे retry करते हैं, जिससे duplicate बन जाते हैं। इसका हल है receiver को idempotent बनाना: processed event IDs track करें और जो पहले handle हो चुके हैं उन्हें skip करें।

6. Practice Exercises

English:

- For (a) uploading a profile photo, (b) multiplayer game position updates, (c) placing a food order, decide TCP or UDP and justify it.
- Draw the WebSocket upgrade handshake from memory.
- Explain why HTTPS doesn't remove the need to think about TCP.

Hindi:

- (a) profile photo upload, (b) multiplayer game के position updates, (c) food order place करना -- इन तीनों के लिए TCP या UDP चुनें और वजह बताएं।
- WebSocket upgrade handshake याद से बनाएं।
- समझाएं कि HTTPS होने पर भी TCP के बारे में सोचना क्यों ज़रूरी रहता है।

7. Curated YouTube Videos (Reference – English)

Language	Video & Channel	Why it helps
English	"TCP vs UDP" — PowerCert Animated Videos	Clean animated breakdown.
English	"WebSockets: Real-Time Communication Explained"	Frames WebSockets for system design interviews.
Hindi	"TCP vs UDP Explained in Hindi"	Direct Hindi comparison with examples.
Hindi	"Lec-77: TCP vs UDP differences in Hindi" — Gate Smashers	Detailed and structured.

8. Key Takeaways

English:

- TCP trades speed for reliability: handshake, acknowledgments, ordered delivery, retransmission.
- UDP trades reliability for speed: no handshake, no guarantees, minimal overhead.
- Choose UDP-based transport for real-time audio/video where a dropped packet is acceptable.
- HTTP/HTTPS is a request-response protocol built on TCP; the client always initiates.
- WebSockets upgrade an HTTP connection into a persistent, full-duplex channel for real-time features.
- Most systems use a mix: HTTPS for standard APIs, WebSockets or UDP only where real-time genuinely matters.

Hindi:

- TCP speed की जगह reliability चुनता है: handshake, acknowledgments, ordered delivery, retransmission।
- UDP reliability की जगह speed चुनता है: कोई handshake नहीं, कोई guarantee नहीं, न्यूनतम overhead।
- Real-time audio/video के लिए UDP-based transport चुनें, जहाँ packet छूटना चल सके।
- HTTP/HTTPS एक request-response protocol है जो TCP पर बना है; client हमेशा शुरुआत करता है।
- WebSockets एक HTTP connection को persistent, full-duplex channel में बदल देते हैं real-time features के लिए।
- ज्यादातर systems मशरूफ use करते हैं: standard APIs के लिए HTTPS, और सच में real-time चाहिए वहीं WebSockets या UDP।

General Words Glossary

A consolidated list of the important English words used throughout this chapter, with Hindi meanings and simple explanations — useful for revising vocabulary after finishing the chapter.

English Word	Hindi Meaning	Simple Explanation
Acknowledged	स्वीकार किया गया	Confirmed as received
Connectionless	कनेक्शनलेस / बिना कनेक्शन के	Not requiring a set-up connection before sending
Encryption	एन्क्रिप्शन / गोपनीयकरण	Scrambling data so only authorized parties can read it
Glitchy	गड़बड़ / अटकने वाला	Occasionally malfunctioning or breaking up
Implicit	अंतर्निहित / परोक्ष	Understood without being directly stated
Laggy	लैगी / धीमा और अटका हुआ	Feeling delayed or not smooth
Multiplexing	मल्टीप्लेक्सिंग	Sending multiple streams of data over one connection
Notify	सूचित करना	To inform someone about something
Overhead	अतिरिक्त बोझ / ओवरहेड	Extra resource cost of doing something
Persistent	स्थायी / लगातार बना रहने वाला	Staying open/active over time
Pragmatic	व्यावहारिक	Practical and sensible, not idealistic
Protocol	प्रोटोकॉल / नियमों का सेट	An agreed set of rules for communication
Reliable	वश्वसनीय / भरोसेमंद	Guaranteed to work correctly every time
Retransmitted	दोबारा भेजा गया	Sent again after being lost